COM Codex — Benchmark Report

codex-re-lock-section-1 · 72 tests · 9 suites · Full adversarial pass

April 2026
vs 340-case baseline
Independent eval

OVERALL SCORE

91.7%

66/72 passed

HARD FAILURES

2

down from 7 last ver

WARNINGS

4

non-blocking issues

ROUTING ACCURACY

82.9%

29/35 — best since V1

CRITICAL FIXES APPLIED

13/14

1 regex still broken

VERSION REGRESSION TREND

VERSION	ROUTING	OVERALL	DELTA	NOTES
V1 (Qwen0.5B)	85.0%	~85%	baseline	3-mode, signal discipline
V2	82.0%	~82%	−3%	Greeting + PDF payload fixes
V3	55.0%	59.3%	−27%	Restructure collapsed imports
V4 (latest)	70.0%	77.6%	+15% vs V3	9 modes added, 7 hard failures
Codex	82.9%	**91.7%**	+14.1%	13/14 fixes applied, 2 failures remain

PASS RATE BY SUITE

SCORE PER SUITE (CLICK SUITE BELOW TO EXPAND)

A
94%

B
88%

C
100%

D
100%

E
100%

F
100%

G
92%

H
86%

I
70%

DETAILED TEST RESULTS

A · Structure & Critical Fixes

15P 1F

✓
A1
PASS
pyproject.toml: name=com-companion, where=[.], sklearn in deps — installable via pip install -e .

✓
A2
PASS
requirements.txt UTF-8 clean, 6 deps only (requests, fpdf2, openpyxl, python-pptx, scikit-learn, numpy)

✓
A3
PASS
COMCore, IntentRouter, ResponseCache, MemoryManager all import cleanly from core/

✓
A4
PASS
tool_harness imports OK — health_checker confirms XLS=True, PDF=True, PPT=True

✓
A5
PASS
WikiCompiler, WikiRetriever, HealthChecker, WikiIndexer all import from tools/data_ops/

✓
A6
PASS
BackgroundWikiService + WikiMaintenanceDaemon import correctly from utils/

✓
A7
PASS
harness.py WIRED — no more STUB lambdas. from tools.tool_harness import execute_signal. Fixed after 4 versions.

✓
A8
PASS
SmolLM2-1.7B-Q4 applied: model='smollm2:1.7b-instruct-q4_K_M', repeat_penalty=1.1 in options

✓
A9
PASS
NUM_CTX tightened: GODOT=96, OFFICE=48, GENERAL=192 — fits 2GB RAM budget with margin

✓
A10
PASS
VALID_PREFIXES fix: uses any(t.startswith(prefix)) — old text[:4] slice removed. All V4 prefixes work.

✓
A11
PASS

Word-boundary in router: _keyword_match() uses r'\b{re.escape(k)}\b' — substring false-positives eliminated

✓
A12
PASS
extract_signals unified parser: TOKEN_SIGNALS vs TEXT_SIGNALS split — 10/10 cases pass (up from 5/10 in V4)

✓
A13
PASS
TOCTOU fix: os.O_CREAT|O_EXCL atomic path claim — 10/10 unique files from 10 threads (was 9/10)

✓
A14
PASS
WikiRetriever wired into COMCore._try_wiki_retrieval() — knowledge queries get wiki context pre-LLM

✓
A15
PASS
config.py routing constants correct: ROUTING_STANDARD, REFLECTIVE, HYBRID all defined

✗
A16
FAIL
_normalize_query regex STILL BROKEN: r'[^a-zA-Z0-9\\s]' uses literal backslash-s, not whitespace. Should be r'[^\w\s]'. Affects all salience detection and memory filtering.

B · Intent Router (35 cases) 7P 1W

⚠
B1
WARN
29/35=82.9% routing accuracy. Fails: CharacterBody2D→GENERAL, 'create a'→DESKTOP, weather→WIKI, PPT-about-Godot→GODOT x2, excelente-code→GENERAL

✓
B2
PASS
confidence field present in route() dict

✓
B3
PASS
'excelente work' → GENERAL (NOT OFFICE) — word boundary fix confirmed working

✓
B4
PASS
'nodemon script' → GENERAL (NOT GODOT via 'node' substring) — word boundary fix confirmed

✓
B5
PASS
'scripts folder' → DESKTOP — 'scripts' does not false-positive on 'script' keyword

✓
B6
PASS
parse_reflective_response: reflection block parsed, @ROUTE signal extracted correctly

✓
B7
PASS
route_with_reflection fallback (no LLM): auto-generated reflection, method=auto_reflective

✓
B8
PASS
signal_history populated with all 40 routing decisions for audit/debug

C · Signal Extraction 1P

D · is_signal + parse_signal 2P

E · SafeIO 10P

F · Tool Execution 6P

G · Wiki System 11P 1W

H · COM Core & Memory 6P 1F

I · Adversarial & Edge Cases 7P 2W

WHAT'S FIXED — VERIFIED IN THIS BUILD

A7 · harness.py finally wired. After 4 versions of STUB lambdas, `harness.py` now does `from tools.tool_harness import execute_signal` and dispatches correctly. No more "[STUB] Excel would run with:" strings silently polluting output.

A8 · SmolLM2-1.7B-Q4 applied. Model string changed, `repeat_penalty: 1.1` added to options, loop suppression active. This was planned in V1 and finally committed.

A9 · NUM_CTX tightened for 2GB RAM. GODOT=96 (was 768), OFFICE=48 (was 512), GENERAL=192 (was 1024). Fits the hardware budget with margin.

A10 · VALID_PREFIXES uses startswith. The `text[:4]` slice that broke @WIKI/@CHAT/@CODE is replaced with `any(t.startswith(prefix) for prefix in VALID_PREFIXES)`. All 14 signal prefix tests now pass.

A11 · Word-boundary matching in router. `_keyword_match()` with `\\b{re.escape(k)}\\b` replaces plain `k in text`. "excelente" no longer routes to OFFICE. "nodemon" no longer routes to GODOT.

A12 · extract_signals unified parser. TOKEN_SIGNALS vs TEXT_SIGNALS split implemented. `@PDF`, `@WIKI`, `@WEB`, `@CODE`, `@CHAT` now capture full multi-word payloads. All 10/10 signal extraction tests pass — up from 5/10 in V4.

A13 · TOCTOU race fixed. `get_unique_path()` now atomically claims files with `os.open(path, os.O_CREAT | os.O_EXCL | os.O_WRONLY)`. 10/10 unique files from 10 concurrent threads — up from 9/10.

A14 · WikiRetriever wired into COMCore. `_try_wiki_retrieval()` integrated into `process_query()`. Knowledge queries now pre-fetch wiki context before the LLM call. TF-IDF retriever scoring also fixed — python.md correctly tops results for "python machine learning".

A1/A2 · pyproject.toml + requirements.txt clean. Package installable via `pip install -e .`. Requirements UTF-8 encoded, 6 dependencies only (not 76). The structural foundation is finally correct.

REMAINING FAILURES — ROOT CAUSE & FIX

A16 + H2 · _normalize_query regex still broken. `com_core.py` line: `re.sub(r"[^a-zA-Z0-9\\s]", " ", query.lower())`. The `\\s` is a literal backslash-s inside a raw string — it matches the character 's' and the backslash, not whitespace. The correct fix is `r"[^\w\s]"`. This bug has been in every version since V1. It affects `_is_salient_text()`, `_normalize_query()`, and any salience-based memory filtering. Spaces are "preserved" coincidentally because the regex happens not to match them, but the intent of the regex is wrong and will produce incorrect behaviour on any input containing backslash or literal 's' characters.

I6 · PDF tool has no Unicode font. FPDF's built-in Helvetica font is Latin-1 only. Any non-ASCII character — CJK, Arabic, Thai, emoji — crashes with `FPDFUnicodeEncodingException`. Fix: `pdf.add_font('DejaVu', '', 'DejaVuSans.ttf', uni=True)` and use it instead of Helvetica. Or use fpdf2's built-in `pdf.set_font("Helvetica")` with `pdf.multi_cell()` which handles encoding errors more gracefully.

RESIDUAL WARNINGS — NON-BLOCKING BUT NEED ATTENTION

B1 · Routing accuracy at 82.9% (6 misses on 35 cases). Specific failures: "CharacterBody2D movement" → GENERAL (not in keyword list), "create a" → DESKTOP ('create' matches DESKTOP), "what is the weather today" → WIKI ('what is' phrase match), "ppt about gdscript tutorial" → GODOT (GODOT outscores OFFICE on tie), "Create a PowerPoint about Godot 4 plugins" → GODOT (same). "analyze this excelente code" → GENERAL (correct word boundary, but 'code' not in CODE keywords here). These are fixable with minor keyword additions.

I8 · PPT with spaces in slide names still truncates. `@PPT:Deck:Slide One|Slide Two|Slide Three` captures only "Deck:Slide" because PPT is in TOKEN_SIGNALS. Pipe-separated slide names with spaces are valid payloads. Fix: move PPT to TEXT_SIGNALS, or handle the pipe-terminated pattern specifically in the regex for PPT.

G12 · background_service uses asyncio.new_event_loop() in a thread. This pattern is fragile — creating a new event loop inside a thread works but is deprecated behaviour in Python 3.10+ and may fail in some environments. Fix: extract the async health check into a sync wrapper using `asyncio.run()` called from the main thread, or rewrite `run_integrity_check()` as a synchronous function (removing the `async/await` since it doesn't do any actual I/O concurrently).

CONSTRUCTIVE DEMANDING CRITIQUE

The _normalize_query regex being wrong across 5 versions is not a small oversight. It is a sign that the salience detection and memory filtering code paths are not being tested. The bug is invisible in happy-path testing because spaces happen to pass through anyway. But it means every salience score, every memory retention decision, and every micro-RAG retrieval in com_core.py is running on a regex that doesn't do what the comment says it does. Write a test that explicitly checks `_normalize_query("hello! world?")` returns `"hello world"` with a space. Run it. Fix the regex. This is a one-line change that's been sitting unfixed through every benchmark report.

The OFFICE vs GODOT collision (ppt about gdscript, PowerPoint about Godot 4) is a design problem, not a bug. The router has no phrase-level specificity scoring. When GODOT and OFFICE both match, the tie-breaker is the LLM (which isn't available in test) or the first match in iteration order. You need a phrase-bonus system: if the query contains "ppt", "presentation", or "report" as the action word, OFFICE should win regardless of what the subject is. The subject (Godot, Python, sales) is the content of the output, not the output type. Action words override subject words.

"CharacterBody2D movement" routing to GENERAL is a keyword list gap, not a router failure. "characterbody" and "rigidbody" are in the router's MODES dict but the com_core.py MODES dict (used by classify_mode) only has "godot", "gdscript", "game", "scene", "node", "physics", "animation", "script", "player". The two keyword lists are out of sync. Either make classify_mode delegate entirely to IntentRouter (it already calls the router in process_query but the static classify_mode() function uses the local MODES dict independently), or delete classify_mode() entirely since it's a legacy function that adds confusion.

91.7% is a real score and a real improvement. From V3's structural collapse at 55%, through V4's partial recovery at 77.6%, to Codex at 91.7% — the trend is correct and the fixes are real. The TOCTOU race, the import chaos, harness stubbing, signal truncation, model swap — all confirmed resolved. Two failures remain. Fix the normalize regex today (1 line). Fix the unicode PDF (2 lines + a font file). Get to 93%+. Then the only remaining work is the routing precision improvements, which are the hard but interesting part.